

Part B: Traditional Text Classification

0. Load and Prepare AG News Dataset

```
In [3]: import pandas as pd
import numpy as np
import time
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from collections import Counter

# Load
train_data = pd.read_csv('ag-news-classification-dataset/train.csv')
test_data = pd.read_csv('ag-news-classification-dataset/test.csv')

target_classes = ['World', 'Sports', 'Business', 'Sci/Tech']

# Concat, convert to lowercase
train_texts = (train_data['Title'] + ' ' + train_data['Description']).str.lower()
test_texts = (test_data['Title'] + ' ' + test_data['Description']).str.lower()

# Labels are 1-index in the dataset, convert to 0-index
train_labels = train_data['Class Index'].values - 1
test_labels = test_data['Class Index'].values - 1

print(f"Training samples: {len(train_texts)}")
print(f"Test samples:      {len(test_texts)}")
print(f"Classes:             {target_classes}")
print(f"\nClass distribution (train): {dict(Counter(train_labels))}")
print(f"Class distribution (test):  {dict(Counter(test_labels))}")
print(f"\nSample text: {train_texts.iloc[0][:100]}...")
```

Training samples: 120000

Test samples: 7600

Classes: ['World', 'Sports', 'Business', 'Sci/Tech']

Class distribution (train): {np.int64(2): 30000, np.int64(3): 30000, np.int64(1): 30000, np.int64(0): 30000}

Class distribution (test): {np.int64(2): 1900, np.int64(3): 1900, np.int64(1): 1900, np.int64(0): 1900}

Sample text: wall st. bears claw back into the black (reuters) reuters - short-sellers, wall street's dwindling\b...

1. Ταξινόμηση

Δημιουργία μονογραμματών και τριγραμματών TF-IDF

```
In [4]: # Word uni-grams TF-IDF
tfidf_word = TfidfVectorizer(analyzer='word', ngram_range=(1, 1))
X_train_word = tfidf_word.fit_transform(train_texts)
X_test_word = tfidf_word.transform(test_texts)

# Character tri-grams TF-IDF
```

```

tfidf_char = TfidfVectorizer(analyzer='char', ngram_range=(3, 3))
X_train_char = tfidf_char.fit_transform(train_texts)
X_test_char = tfidf_char.transform(test_texts)

print(f"Word uni-gram features (dimensionality): {X_train_word.shape[1]}")
print(f"Char tri-gram features (dimensionality): {X_train_char.shape[1]}")

```

Word uni-gram features (dimensionality): 64999
Char tri-gram features (dimensionality): 31074

Εκπαίδευση και Evaluation των τεσσάρων μοντέλων

```

In [5]: results = {}
all_preds = {}

models_config = [
    ('MNB + Word Uni-grams', MultinomialNB(), X_train_word, X_test_word, X_train_word.shape[1]),
    ('MNB + Char Tri-grams', MultinomialNB(), X_train_char, X_test_char, X_train_char.shape[1]),
    ('SVM + Word Uni-grams', LinearSVC(C=1, max_iter=10000), X_train_word, X_test_word, X_train_word.shape[1]),
    ('SVM + Char Tri-grams', LinearSVC(C=1, max_iter=10000), X_train_char, X_test_char, X_train_char.shape[1]),
]

for name, model, X_tr, X_te, n_features in models_config:
    print(f"\n{'='*60}")
    print(f"  Training: {name}")
    print(f"{'='*60}")

    start_time = time.time()
    model.fit(X_tr, train_labels)
    preds = model.predict(X_te)
    elapsed = time.time() - start_time

    acc = accuracy_score(test_labels, preds)
    all_preds[name] = preds

    results[name] = {
        'accuracy': acc,
        'dimensionality': n_features,
        'time': elapsed
    }

    print(f"  Accuracy:      {acc:.4f}")
    print(f"  Dimensionality: {n_features:,}")
    print(f"  Time:           {elapsed:.2f}s")
    print(f"\n  Classification Report:")
    print(classification_report(test_labels, preds, target_names=target_classes))

```

```

=====
Training: MNB + Word Uni-grams
=====
Accuracy:      0.9022
Dimensionality: 64,999
Time:          0.05s

Classification Report:
      precision    recall  f1-score   support

   World         0.91      0.89      0.90      1900
   Sports         0.95      0.98      0.96      1900
  Business         0.87      0.86      0.86      1900
  Sci/Tech         0.88      0.88      0.88      1900

 accuracy          0.90          0.90      0.90      7600
  macro avg         0.90      0.90      0.90      7600
weighted avg         0.90      0.90      0.90      7600

```

```

=====
Training: MNB + Char Tri-grams
=====
Accuracy:      0.8687
Dimensionality: 31,074
Time:          0.09s

Classification Report:
      precision    recall  f1-score   support

   World         0.87      0.89      0.88      1900
   Sports         0.90      0.96      0.93      1900
  Business         0.86      0.79      0.82      1900
  Sci/Tech         0.84      0.84      0.84      1900

 accuracy          0.87          0.87      0.87      7600
  macro avg         0.87      0.87      0.87      7600
weighted avg         0.87      0.87      0.87      7600

```

```

=====
Training: SVM + Word Uni-grams
=====
Accuracy:      0.9196
Dimensionality: 64,999
Time:          9.73s

Classification Report:
      precision    recall  f1-score   support

   World         0.93      0.90      0.92      1900
   Sports         0.96      0.98      0.97      1900
  Business         0.89      0.89      0.89      1900
  Sci/Tech         0.90      0.90      0.90      1900

 accuracy          0.92          0.92      0.92      7600
  macro avg         0.92      0.92      0.92      7600
weighted avg         0.92      0.92      0.92      7600

```

```
=====
Training: SVM + Char Tri-grams
=====

Accuracy:      0.9120
Dimensionality: 31,074
Time:          27.79s

Classification Report:
      precision    recall  f1-score   support

   World       0.93      0.90      0.92      1900
   Sports       0.95      0.97      0.96      1900
 Business       0.88      0.88      0.88      1900
  Sci/Tech       0.89      0.89      0.89      1900

 accuracy          0.91          0.91          0.91          7600
 macro avg          0.91          0.91          0.91          7600
weighted avg          0.91          0.91          0.91          7600
```

2. Συμπλήρωση Πίνακα

```
In [ ]: # summary table
print(f"{'Model':<30s} {'Accuracy':>10s} {'Dimensions':>12s} {'Time (s)':>10s}")
print('-' * 65)
for name, r in results.items():
    print(f"{'name':<30s} {r['accuracy']:>10.4f} {r['dimensionality']:>12,} {r['time']:>10.2f}")
```

Model	Accuracy	Dimensions	Time (s)

MNB + Word Uni-grams	0.9022	64,999	0.05
MNB + Char Tri-grams	0.8687	31,074	0.09
SVM + Word Uni-grams	0.9196	64,999	9.73
SVM + Char Tri-grams	0.9120	31,074	27.79

Πίνακας Εκφώνησης

	NB (word 1-grams)	NB (char 3-grams)	SVM (word 1-grams)	SVM (char 3-grams)

Accuracy	0.9022	0.8687	0.9196	0.9120
Dimensionality	64,999	31,074	64,999	31,074
Time cost	0.05s	0.09s	9.73s	27.79s

3. Λάθος ταξινομημένα παραδείγματα από όλα τα μοντέλα

```
In [11]: model_names = list(all_preds.keys())
all_wrong_mask = np.ones(len(test_labels), dtype=bool)

for name in model_names:
    preds = all_preds[name]
    all_wrong_mask &= (preds != test_labels)

wrong_indices = np.where(all_wrong_mask)[0]
print(f"Total examples misclassified by ALL 4 models: {len(wrong_indices)}")
```

```
# Counts ana katigoria
wrong_true_labels = test_labels[wrong_indices]
print(f"\nMisclassified count per category:")
for cat_idx, cat_name in enumerate(target_classes):
    count = np.sum(wrong_true_labels == cat_idx)
    print(f" {cat_name}: {count}")
```

Total examples misclassified by ALL 4 models: 341

Misclassified count per category:

```
World: 112
Sports: 9
Business: 135
Sci/Tech: 85
```

Παρατηρήσεις

Όπως φαίνεται παραπάνω, τα παραδείγματα που κατηγοριοποιήθηκαν λάθος ανοίκουν κατά συντριπτική πλειοψηφία στις κλάσεις World και Business. Εκ φύσης τους, αυτές οι 2 κατηγορίες είναι πολύ ευρείες και αόριστες, οπότε το φαινόμενο αυτό είναι αναμενόμενο.

```
In [10]: # Print one specific example
if len(wrong_indices) > 0:
    idx = wrong_indices[0]
    print(f"Example index: {idx}")
    print(f"True label:    {target_classes[test_labels[idx]]}")
    print(f"Text:         {test_texts.iloc[idx]}")
    print(f"\nPredictions by each model:")
    for name in model_names:
        print(f" {name:<30s} -> {target_classes[all_preds[name][idx]]}")
else:
    print("No examples were misclassified by all 4 models.")
```

Example index: 24

True label: Sci/Tech

Text: rivals try to turn tables on charles schwab by michael liedtke san francisco (ap) -- with its low prices and iconoclastic attitude, discount stock broker charles schwab corp. (sch) represented an annoying stone in wall street's wing-tipped shoes for decades...

Predictions by each model:

```
MNB + Word Uni-grams      -> Business
MNB + Char Tri-grams      -> Business
SVM + Word Uni-grams      -> Business
SVM + Char Tri-grams      -> Business
```

Παρατίθεται το πλήρες περιεχόμενο του mislabelled κειμενου που ανασύρθηκε:

"rivals try to turn tables on charles schwab by michael liedtke san francisco (ap) -- with its low prices and iconoclastic attitude, discount stock broker charles schwab corp. (sch) represented an annoying stone in wall street's wing-tipped shoes for decades..."

Βάσει του παραπάνω, γίνεται αντιληπτό πως σε κάποια δείγματα είναι τέτοιο το περιεχόμενο που ακόμα και ένας άνθρωπος ταξινομητής θα μπορούσε εύκολα να τα κατατάξει ως Business αντί για sci/tech Κ.Ο.Κ.